Nexus Commerce — Documentação do Projeto

Oct 11, 2026 · @diogo

Visão geral

O Nexus Commerce é uma plataforma de processamento de pedidos com arquitetura de microsserviços, feita em .NET e RabbitMQ, que simula o funcionamento interno de uma loja online.

O tema é a **Nexus Commerce**, uma loja de roupas que vende camisetas, bonés e moletons. O tema é simples de propósito, para o foco ficar na arquitetura e não em regras de negócio complexas.

Objetivo: projeto de portfólio para GitHub e LinkedIn, mostrando que o desenvolvedor sabe construir serviços que se comunicam por eventos, lidam com falhas e podem crescer, e não apenas APIs CRUD.

Por que microsserviços: é um tema frequente nas vagas de desenvolvedor .NET e um assunto comum em entrevistas técnicas.

Escopo

A primeira versão (MVP) precisa provar uma coisa: os pedidos são processados corretamente e os serviços se comunicam com segurança por mensagens.

Entra no MVP

- Serviço de **Pedidos**: cria pedidos e controla o status (Pendente, Confirmado, Recusado)
- Serviço de **Estoque**: cadastra produtos e reserva quantidades
- Serviço de **Notificações**: registra os avisos enviados ao cliente
- Comunicação assíncrona entre os serviços via RabbitMQ
- Execução local de tudo com Docker Compose

Fica para versões futuras

- Pagamento real, cupons e carrinho completo
- Autenticação (JWT) e API Gateway
- Kubernetes
- Catálogo completo de e-commerce

Requisitos

Requisitos funcionais

ID	Requisito	Serviço
RF01	O cliente pode criar um pedido com um ou mais produtos e suas quantidades	Pedidos
RF02	Todo pedido novo é registrado com status Pendente	Pedidos
RF03	O cliente pode consultar um pedido e o status atual dele	Pedidos
RF04	O administrador pode cadastrar produtos e definir a quantidade em estoque	Estoque
RF05	O sistema verifica se há estoque suficiente para todos os itens do pedido	Estoque
RF06	Havendo estoque, o sistema reserva as quantidades e o pedido passa para Confirmado	Estoque + Pedidos
RF07	Faltando estoque, nada é reservado e o pedido passa para Recusado	Estoque + Pedidos
RF08	O cliente recebe um aviso quando o pedido é confirmado ou recusado	Notificações
RF09	É possível consultar o histórico de notificações de um pedido	Notificações

Requisitos não funcionais

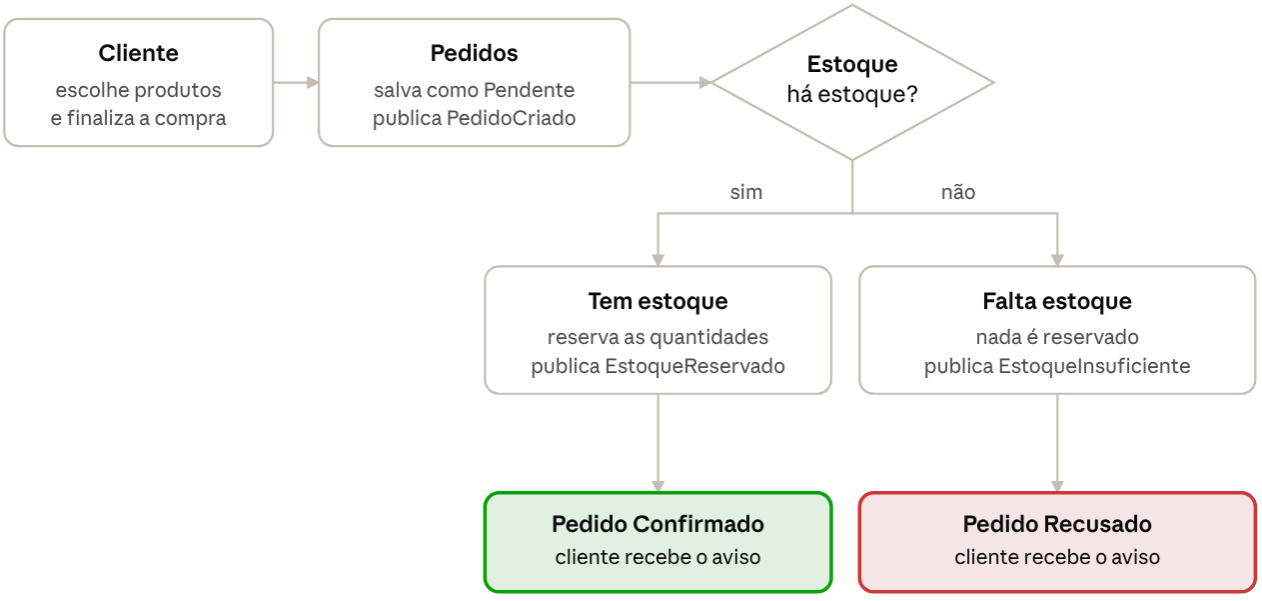
ID	Requisito
RNF01	Cada serviço é uma aplicação .NET independente, com banco de dados próprio
RNF02	Os serviços se comunicam por eventos no RabbitMQ; um serviço nunca acessa o banco de outro
RNF03	A criação do pedido responde sem esperar o processamento de estoque e notificação

ID	Requisito
RNF04	Uma mensagem repetida não pode reservar estoque duas vezes (idempotência)
RNF05	Mensagens que falham são retentadas e, depois de várias falhas, vão para uma fila de erro (DLQ)
RNF06	O sistema inteiro sobe com um único comando (docker compose up)
RNF07	Os logs permitem rastrear o processamento de um pedido do início ao fim
RNF08	As APIs são documentadas e testáveis pelo Swagger

Fluxo principal do pedido

Todo pedido termina Confirmado ou Recusado, e o serviço de Pedidos nunca consulta o banco do Estoque: tudo passa por eventos no RabbitMQ.

O pedido termina Confirmado ou Recusado, conforme o estoque



fluxo do pedido · 1 decisão, 2 resultados

O RabbitMQ não atualiza o pedido sozinho. O Estoque publica o resultado, e um consumidor do serviço de Pedidos recebe esse evento e atualiza o status no banco. O serviço de Notificações consome os mesmos eventos e registra o aviso ao cliente.

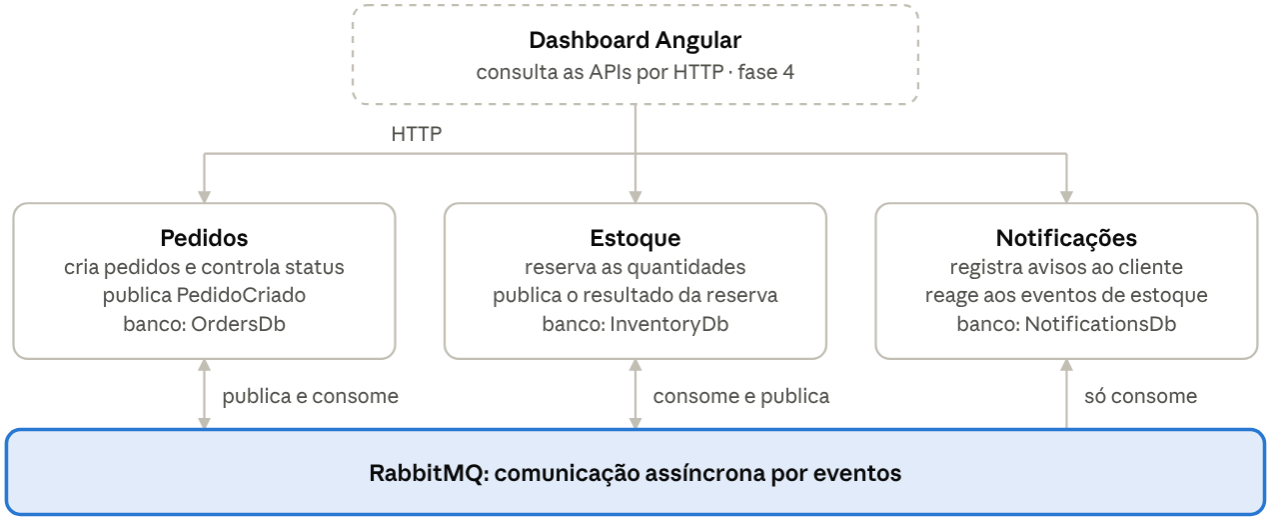
Exemplo concreto

1. Estoque inicial: 5 camisetas
2. Pedido A pede 3 camisetas: estoque reservado, pedido Confirmado
3. Pedido B pede 4 camisetas: restam só 2, pedido Recusado
4. O cliente acompanha o status de cada pedido

Arquitetura

São três serviços .NET independentes, cada um com seu próprio banco SQL Server, que se comunicam apenas por eventos no RabbitMQ.

Os serviços só conversam pelo RabbitMQ, e cada um tem seu próprio banco



arquitetura · 3 serviços, 1 broker

O dashboard (tracejado) entra só na fase 4. Antes disso, as APIs são testadas pelo Swagger.

Eventos

Evento	Publicado por	Consumido por	Efeito
PedidoCriado	Pedidos	Estoque	Estoque tenta reservar os itens
EstoqueReservado	Estoque	Pedidos, Notificações	Pedido vira Confirmado e o cliente é avisado
EstoqueInsuficiente	Estoque	Pedidos, Notificações	Pedido vira Recusado e o cliente é avisado

Cada consumidor tem sua própria fila. Uma mensagem que falha várias vezes vai para a fila de erro (DLQ), em vez de travar as outras.

Stack de tecnologias

A stack reaproveita o que já é usado no estágio e no BarberFlow, para o esforço ir para arquitetura e mensageria.

Tecnologia	Uso no projeto	Fase
C# / ASP.NET Core	APIs dos microsserviços	1
Entity Framework Core	Acesso aos bancos de dados	1
SQL Server	Um banco por serviço	1
Swagger	Documentação e testes manuais das APIs	1
RabbitMQ	Comunicação assíncrona entre os serviços	2
.NET Worker Service	Consumidores de mensagens em segundo plano	2
xUnit	Testes automatizados	3
Angular	Dashboard de pedidos, estoque e notificações	4
Docker / Docker Compose	Infra local (SQL Server, RabbitMQ) desde o início; serviços containerizados na fase 5	1 e 5
GitHub Actions	Build e testes automáticos (CI)	5
OpenTelemetry	Rastreamento e observabilidade	Futuro

Kubernetes fica de fora: Docker Compose é suficiente até o projeto justificar mais infraestrutura.

Estrutura do repositório

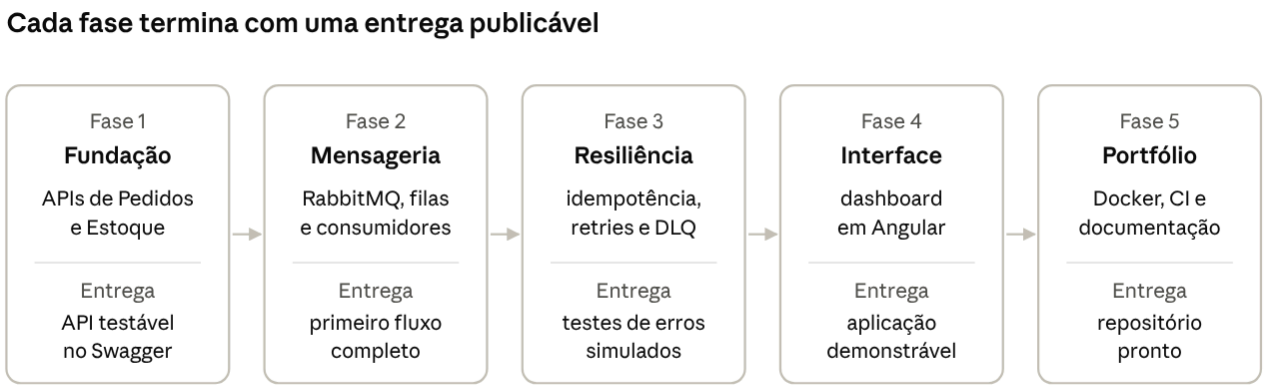
Um único repositório (monorepo) com todos os serviços, para quem avalia o projeto conseguir clonar e rodar tudo com um comando.

```
nexus-commerce/
├── docs/
│   ├── requisitos.md
│   └── arquitetura.png
├── src/
│   ├── Orders/           # API + consumidor de eventos de Pedidos
│   ├── Inventory/        # API + consumidor de eventos de Estoque
│   ├── Notifications/    # Worker de Notificações
│   ├── Shared/           # Contratos dos eventos (PedidoCriado etc.)
│   └── web/              # Dashboard Angular (fase 4)
├── tests/
├── docker-compose.yml
└── README.md
```

O projeto Shared guarda só os contratos das mensagens. Regras de negócio nunca são compartilhadas entre serviços.

Roadmap

O projeto avança em cinco fases, e cada uma termina com uma entrega que pode virar post. As datas ainda não foram definidas.



roadmap · 5 fases, sem datas definidas

A fase 1 é feita sem RabbitMQ de propósito: com as APIs funcionando e testadas, a mensageria entra sobre uma base estável. O SQL Server e o RabbitMQ rodam em containers desde o início; na fase 5 os próprios serviços são containerizados.

Plano de conteúdo para o LinkedIn

Publicar durante o desenvolvimento, mostrando decisões e aprendizados reais, em vez de esperar o projeto ficar pronto.

Post	Quando	Conteúdo
1. Apresentação	Após a documentação	O que é o Nexus Commerce, objetivo e diagrama de arquitetura
2. O primeiro evento	Fim da fase 2	Como um pedido publicado no RabbitMQ chega ao Estoque e o resultado volta para Pedidos
3. E quando algo falha?	Fim da fase 3	Serviço fora do ar, mensagem repetida e como o sistema reage
4. Resultado final	Fim da fase 5	Vídeo curto do fluxo no dashboard + link do GitHub

O README deve ter três demonstrações obrigatórias:

- Um pedido confirmado porque havia estoque
- Um pedido recusado por falta de estoque
- Uma falha simulada, com evidência de que o sistema tratou ou recuperou o processamento

Decisões em aberto

- [ ] Tamanho do projeto: enxuto (só fases 1 e 2), profissional (fases 1 a 5) ou avançado (com observabilidade e CI/CD completos). Este documento assume o profissional.
- [ ] Tema confirmado: loja de roupas (Nexus Commerce) ou outro, como produtos de barbearia
- [ ] Biblioteca de mensageria: cliente oficial do RabbitMQ (mais didático) ou MassTransit (mais usado no mercado)
- [ ] Notificações no MVP: só registro em banco/log ou envio de e-mail de teste